

CY-321 | SECURE SOFTWARE DESIGN

# SECURE QR-BASED ATTENDANCE MANAGEMENT SYSTEM

AWAIS KHAN (2023139)  
MOAWIZ-UR-REHMAN (2023278)  
ZULQARNAIN UMAR (2023556)

# THE PROBLEM

- Traditional attendance methods have critical flaws.
- Proxy Attendance – One student marks attendance for an absent friend. No verification of who is actually present.
- Screenshot Sharing – QR codes get photographed and shared via WhatsApp. Remote students can scan from home.
- Replay Attacks – Captured QR data can be stored and resubmitted later. The server cannot tell the difference.
- No Proof – When disputes happen, there is no cryptographic evidence of who actually scanned what.
- The Result: Attendance records cannot be trusted for audits, financial aid, or student safety.



# OUR SOLUTION

A unified web application with three portals: Teacher, Student, and Admin.

## How It Works:

- A teacher logs in and starts an attendance session. The server generates a QR code that refreshes every fifteen seconds. Each QR contains a unique cryptographic signature that cannot be forged.
- A student scans the code with their phone camera. The server runs eight security checks before marking attendance.
- The teacher watches live attendance numbers and locks the session when class ends. Once locked, no further changes are possible.

**The Key:** A QR code is only valid for fifteen seconds. A screenshot becomes useless almost instantly.



# FOUR SECURITY LAYERS

## Layer One – HMAC-Signed QR Codes

Every QR payload is cryptographically signed. If anyone modifies any field, the signature breaks and the request is rejected. This blocks tampering.

## Layer Two – Single-Use Nonces with 15-Second Expiry

Each QR contains a completely random nonce that can only be used once. The server stores every used nonce forever. Screenshot sharing and replay attacks are completely blocked.

## Layer Three – Device Fingerprinting

The first scan captures a unique fingerprint of the student's device. That fingerprint gets permanently bound to their account. Future scans must come from the exact same device. This blocks proxy attendance.

## Layer Four – Secure JWT Cookies

Authentication tokens are stored in HTTP-only cookies that JavaScript cannot read. This makes token theft via XSS attacks impossible.



# THREAT MODEL RESULTS

We used Microsoft's STRIDE framework to identify every possible threat.

## **Spoofing (impersonation)**

Mitigated by device binding and bcrypt passwords.

## **Tampering (data modification)**

Mitigated by HMAC signatures on every QR.



## **Repudiation (denying actions)**

Mitigated by complete audit trails with timestamps and device IDs.

## **Information Disclosure (data leaks)**

Mitigated by RBAC, secure cookies, and HTTPS.



## **Denial of Service (system flooding)**

Mitigated by rate limiting on all endpoints.

## **Elevation of Privilege (role escalation)**

Mitigated by server-side role verification from database.



# TESTING RESULTS



We tried to break the system. Every attack failed.

**Replay Attack** – Captured a valid QR and resubmitted it seconds later. Server blocked it with a nonce already used error.

**QR Tampering** – Modified a timestamp and kept the original signature. Server rejected it with signature verification failed.

**Authentication Bypass** – Used a student token to call a teacher endpoint. Server returned access denied.

**Privilege Escalation** – Changed the role field inside a JWT from student to teacher. Server ignored the JWT role and checked the database instead. Access denied.

**SQL Injection** – Submitted classic injection payloads in every form field. Parameterized queries treated all input as plain text. No injection succeeded.

**Device Mismatch** – Bound an account to one device, then tried scanning from a different device. Server returned device mismatch error.

# OWASP ZAP RESULTS



We ran an automated security scan using industry-standard OWASP ZAP.

- High Risk Vulnerabilities: Zero
- Medium Risk Alerts: Five, all configuration issues
- Low Risk Alerts: Three, all informational

## What Were the Medium Risk Issues?

Missing security headers like CSP, HSTS, and X-Frame-Options. The server also revealed it was running Express through the X-Powered-By header. CORS was set to allow any domain.

## How We Fixed Them

All security headers were added to the Express middleware. The X-Powered-By header was removed. CORS was restricted to trusted origins only.





# RISK REDUCTION

We measured risk as likelihood multiplied by impact, scoring from one to twenty-five.

## **Before Mitigation:**

QR replay attacks scored twenty out of twenty-five – critical. Authentication bypass scored fifteen. Several other risks scored in the high range.

Our overall risk posture measured seventy point six percent – **HIGH risk.**

## **After Mitigation:**

QR replay attacks dropped to a score of two. Authentication bypass dropped to three. Every other risk dropped significantly.

Our overall risk posture now measures nineteen point four percent – **LOW risk.**

We reduced total risk by over fifty percentage points.

**What We Accept:** Zero-day vulnerabilities in third-party libraries. No system can prevent unknown threats. We monitor security advisories and patch critical issues within forty-eight hours.





# WHAT MAKES THIS DIFFERENT

This is not just another QR attendance app.

- Paper roll call has no security at all. Anyone can sign any name. Records can be altered. There is no audit trail.
- Static QR codes can be photographed and shared. A student sitting at home can scan a friend's screenshot. Replay attacks work perfectly.
- Existing digital systems often store tokens where JavaScript can steal them. Many trust the role inside the JWT, allowing privilege escalation.

Our system has fifteen second QR refresh so screenshots are useless. Single-use nonces so replay attacks never work. Device binding so proxy attendance is impossible. HTTP-only cookies so JavaScript cannot steal tokens. Server-side role verification so JWT tampering does nothing.



# CONCLUSION

## What We Achieved:

- All six STRIDE threat categories fully mitigated.
- Every penetration test attack was successfully blocked.
- OWASP ZAP scan found zero high-risk vulnerabilities.
- Overall risk reduced from seventy point six percent high to nineteen point four percent low.

## The System Is:

- Secure – Multiple independent layers protect against every attack we could simulate.
- Verifiable – Every security claim is backed by test results.
- Production-Ready – Deployed and working on real infrastructure.

**SECURE SOFTWARE DESIGN**



**THANKYOU!**

**AWAIS KHAN**

**U2023139@GIKI.EDU.PK**

**MOAWIZ**

**U2023278@GIKI.EDU.PK**

**ZULQARNAIN UMAR**

**U2023556@GIKI.EDU.PK**